

# InfraFleet

*Piotr Kniat*

## 1. Business description of the system

**InfraFleet** is a modern web application designed for centralized IT infrastructure management. The system acts as a central registry for all servers and workstations within an organization, and also allows for the remote execution of defined administrative tasks on connected machines.

In growing IT environments, systems administrators and Helpdesk technicians often struggle with decentralized documentation (e.g. inventory scattered across spreadsheets) and the need to log into multiple machines separately to perform routine tasks (e.g. system updates, service restarts). The lack of a cohesive system slows down operations and increases the risk of human error. InfraFleet is created to automate these processes, reduce incident response times, and provide Role-Based Access Control (RBAC) to critical network resources.

### **Main features:**

- Infrastructure inventory: Adding, editing, and removing IT assets such as IP addresses, operating systems, machine statuses.
- Task execution management: Ability to run predefined commands or automation scripts (e.g. integration with tools like Ansible or SaltStack) directly from the web browser.
- History and audit Logs: A registry of executed tasks along with logs to facilitate troubleshooting.
- Role-Based Access Control (RBAC): Separation into roles (e.g. SuperAdmin, Helpdesk) where lower-tier employees can only execute safe, predefined tasks without root access.

## 2. Software requirements specification (SRS)

### **Actors in the system:**

- Admin (Super Administrator): Full access to the entire system, user management.
- Technician (IT/Helpdesk Employee): Read-only access to the hardware inventory and the ability to run tasks assigned to their role.

## Functional Requirements (User Stories)

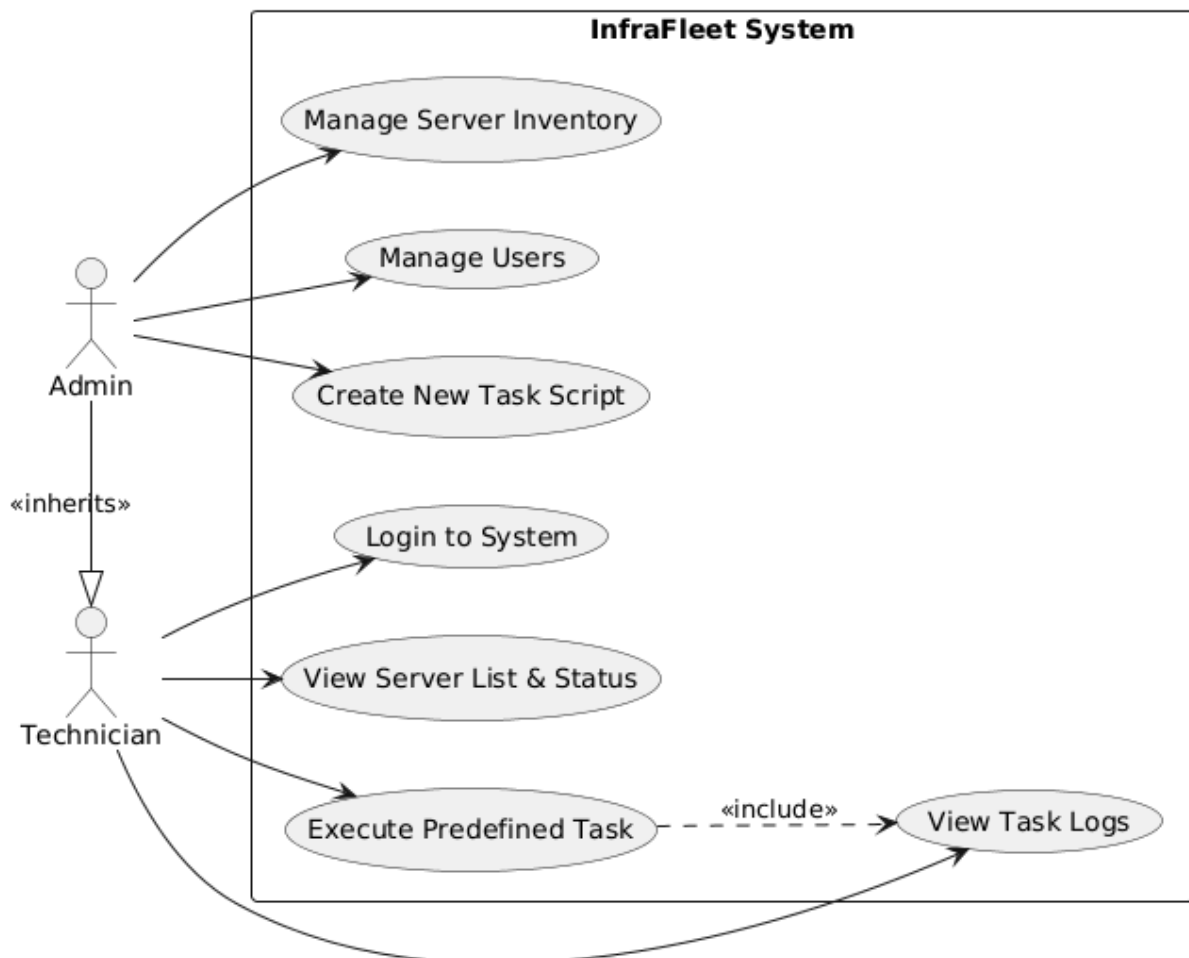
| Actor        | User Story                                                                                                                                   | Acceptance Criteria                                                                                   |
|--------------|----------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Admin / Tech | As a user, I want to log into the system so that I can access the dashboard.                                                                 | System validates login and password in the database. Successful login redirects to the dashboard.     |
| Admin        | As an admin, I want to add a new server to the database so that I have an up-to-date inventory.                                              | The form requires providing the Name, IP Address, and selecting the Operating System.                 |
| Admin        | As an admin, I want to create accounts for Helpdesk employees so that I can grant them access.                                               | Ability to define a role (Admin/Tech) for the new user.                                               |
| Admin / Tech | As a user, I want to view a list of all machines and their status (Online/Offline) so that I can quickly locate failures.                    | Table with filtering capabilities by name, operating system, or status.                               |
| Tech         | As a technician, I want to run a predefined task (e.g. "Restart Nginx service") on a selected server so that I don't have to log in via SSH. | After clicking "Run", the system sends the task and disables the button until the result is received. |
| Admin / Tech | As a user, I want to review the logs from task execution so that I can verify its success.                                                   | The task history panel shows the timestamp, author, status, and console output.                       |

## Non-functional Requirements

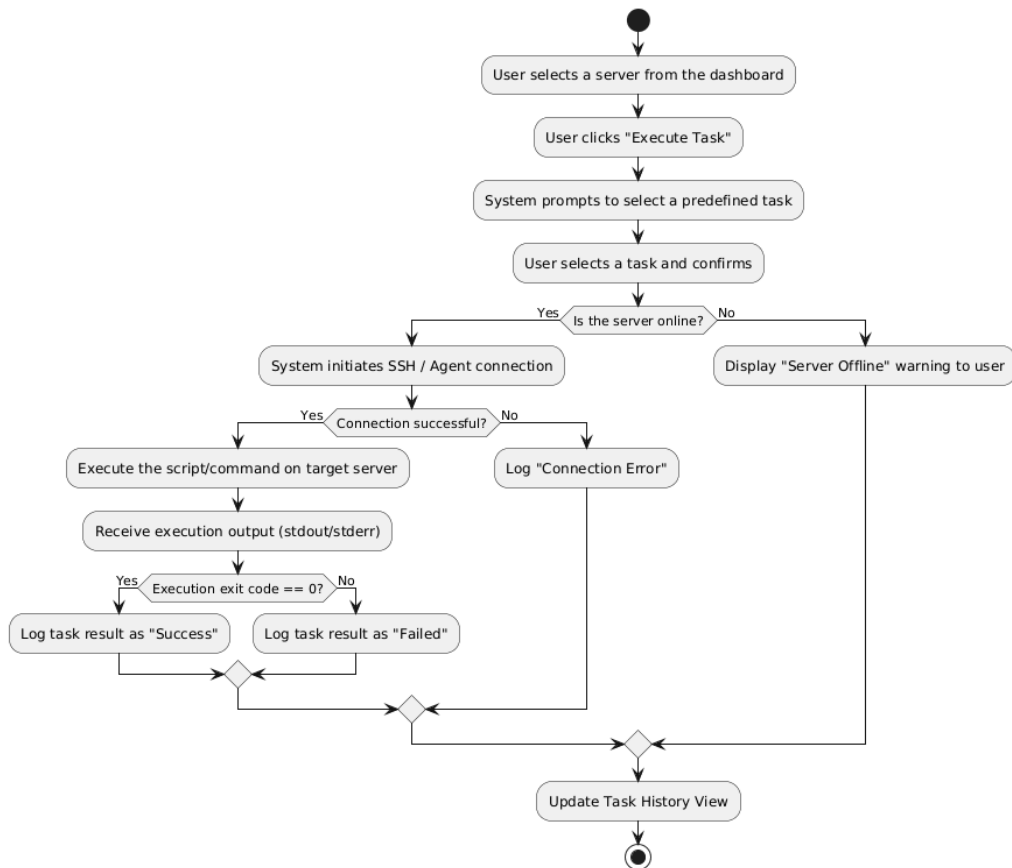
- Security: The system must require authentication for every view (except the login page). User passwords must be stored in the database in a hashed format.
- Architecture: The application must be implemented using a client-server architecture. The web user interface communicates with the backend server via a REST API.
- Performance: The dashboard displaying the list of servers (up to 500 records) must load and render in under 2 seconds.
- Usability: The user interface must be responsive and readable on screens with a minimum resolution of 1280x720px.
- Reliability: A script execution error on a target server must not cause the central application itself to freeze or crash.

## 3. Models of projected system

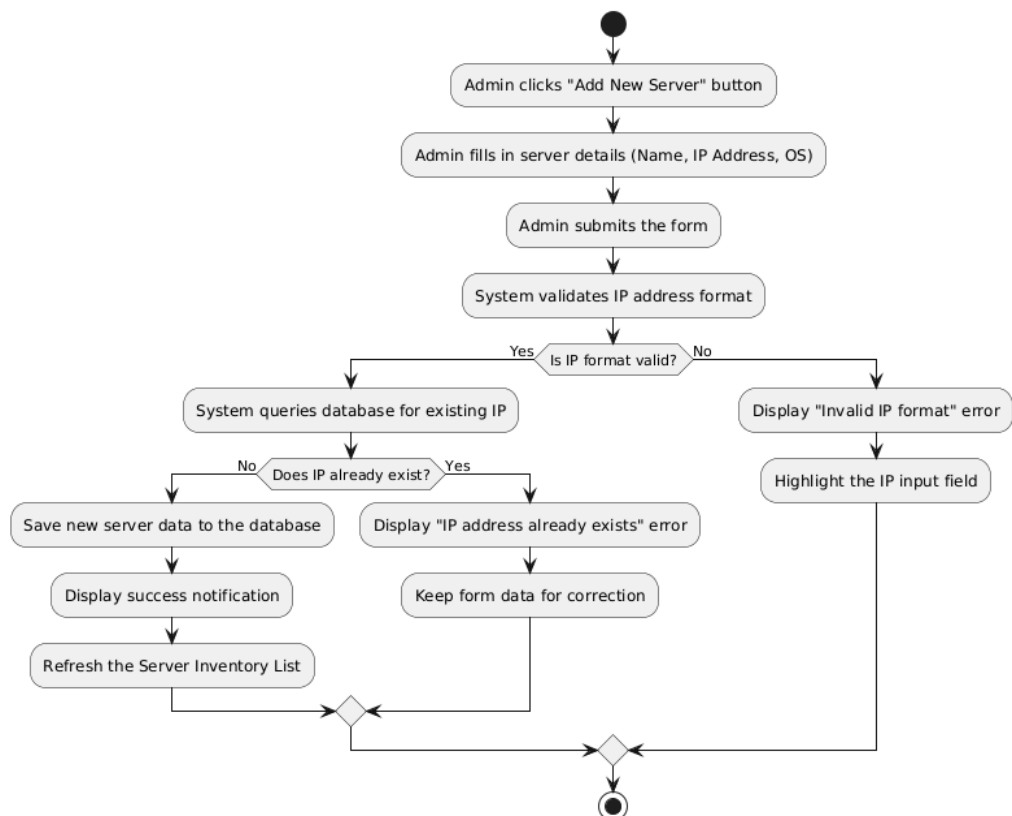
- Use case diagram



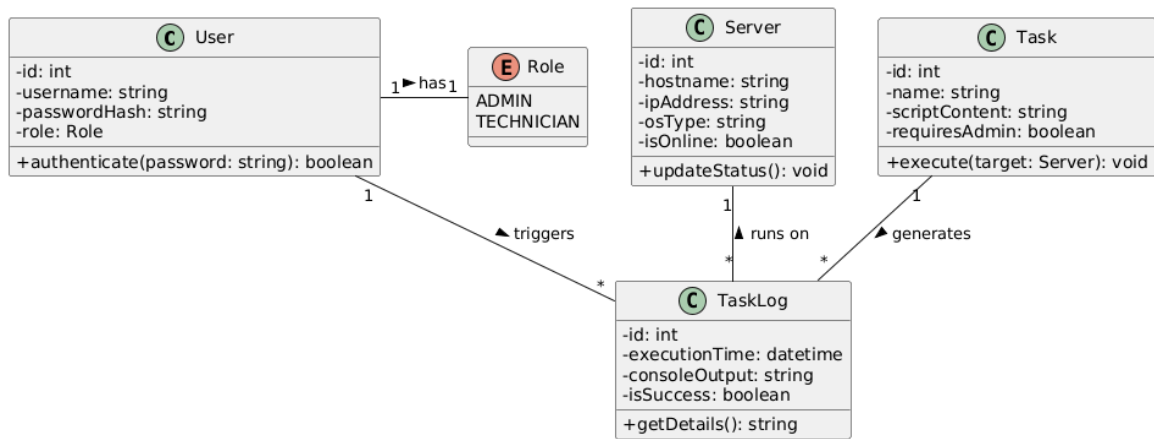
- Activity diagram: executing a task on a server



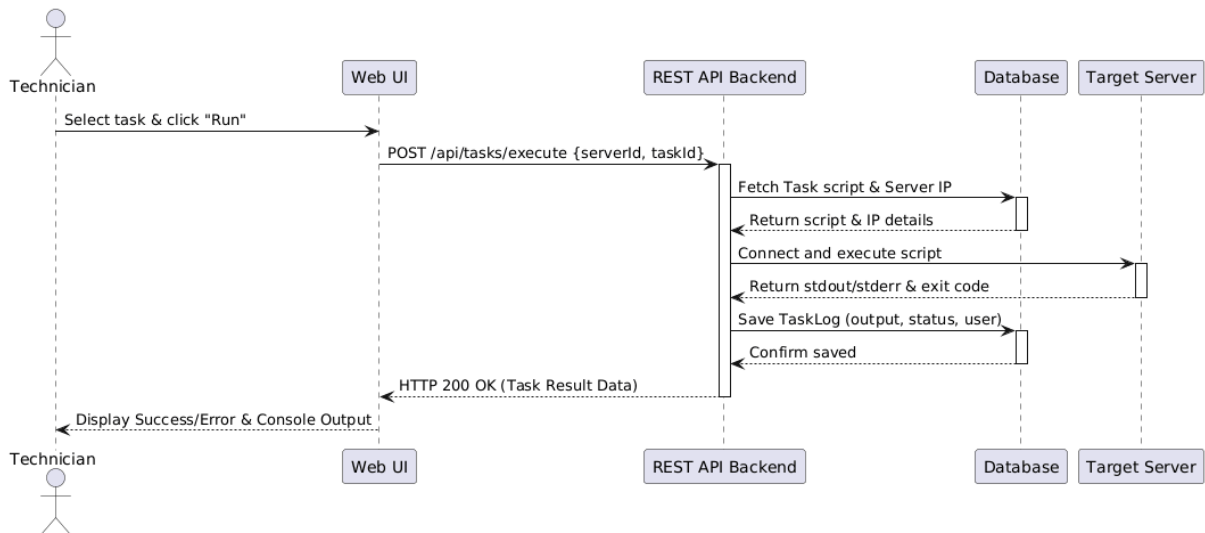
- Activity diagram: adding a new server to inventory



- Class diagram



- Sequence diagram



## 4. Technology Stack and Implementation

The system prototype was implemented as a Static Web Mockup. To ensure rapid prototyping and easy review without complex environment setup, the current version focuses purely on the User Interface and frontend logic, simulating backend interactions.

### Frontend:

- Markup & Styling: HTML5 and CSS3.
- CSS Framework: Bootstrap 5. Integrated via CDN to quickly develop a modern, clean, and Responsive Web Design. It ensures the dashboard looks professional and scales correctly on different screen sizes.
- Interactivity & Logic: Vanilla JavaScript. It handles user interactions and simulates asynchronous backend processes. For example, it dynamically updates the task

execution console and mimics server response delays using `setTimeout()`, providing a realistic feel of the application without needing a real server.

**Backend and Data Storage:**

- Prototyping Data Source: For this prototype phase, the backend server and database were intentionally bypassed. All server inventory data is mocked and hardcoded directly into the frontend views.
- Note for future development: In a fully deployed production version, this static frontend would be integrated with a REST API backend (e.g. Python FastAPI) connected to a relational database (e.g. PostgreSQL).